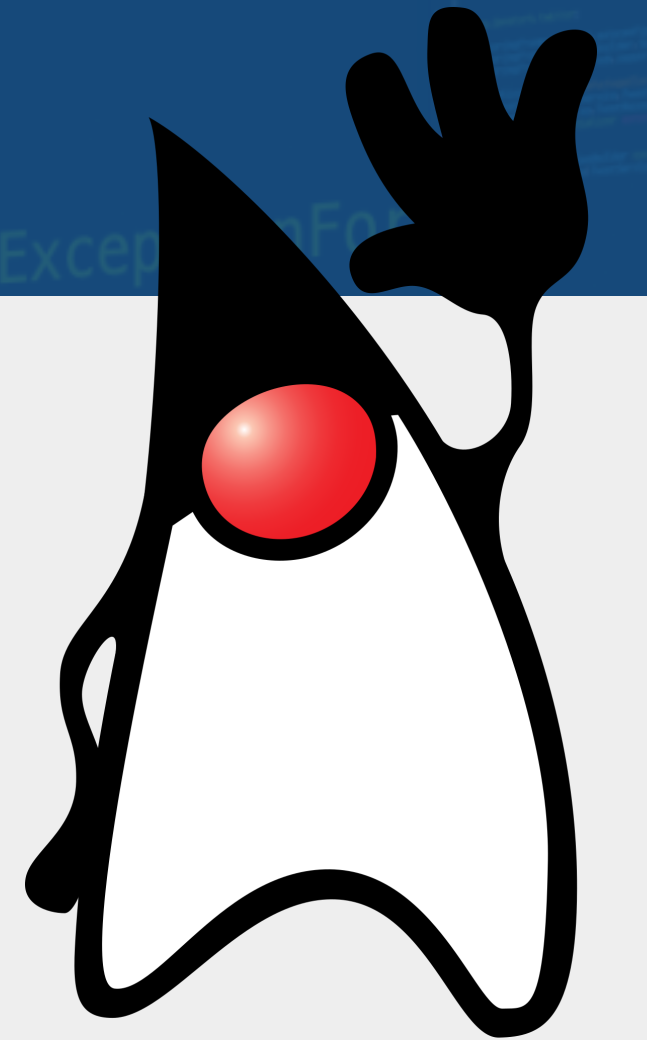


Java Programlama Dili

18. Bölüm static

Akın Kaldıroğlu



Amaç



- Bu bölümde `static` anahtar kelimesi ele alınacaktır



- **static**
 - static Alanlar
 - static Metotlar
- **Ne Zaman static?**
 - Araçsal Sınıflar
- **Ne Zaman static Değil?**
 - static Bulaşıcıdır
- **Java'da Sabiteler**
- **Not**
- **Ödevler**
- **Bundan Sonra?**

static



static



- `static` bir anahtar kelimedir ve alan ve metotları niteler
- `static` alan ve metotlar, nesneler için değil sınıflar için tanımlanır
- Şimdi bunun ne anlama geldiğini görelim



static Alanlar

Nesnelerin Durumu



- Her nesnenin bellekte kendi durumunu ifade eden değişkenlere sahip olduğunu ve bu değişkenlerin değerinin diğer nesnelerdeki değişkenlerin değerinden bağımsız olduğunu biliyoruz
- Nesne değişkeni (instance variable) ya da alan (field) denilen bu özellikler nesneye hastır, muhtemelen her nesne için de farklı değer alırlar ve diğer nesnelerdeki alanlardan bağımsız olarak değişirler

Nesnelerin Ortak Durumu



- Bazen nesnelerin öyle özellikleri olur ki değeri nesneden nesneye değişmez, aynı sınıftan oluşturulmuş tüm nesneler için aynıdır
- Bu durumda o veriyi, her nesnede ayrı ayrı nesne değişkeni olarak tanımlamak uygun değildir
- Çünkü aslen bir tane değişken ve tek bir değer yeterlidir
- Her nesne için ayrı ayrı tutulması hem bellek israfıdır hem de değişim halinde tüm nesnelerin üzerindeki değeri değiştirmek gerekir

Sınıf Değişkeni - I



- Değeri nesneden nesneye değişmeyip her nesne için aynı olan değişkene **sınıf değişkeni (class variable)** denir
- Nesne değişkeni her nesne için ayrı ayrı oluşturulurken, sınıf değişkeni tüm nesnelerin ortak bir durumu olarak bir kere oluşturulur ve sınıfın üzerinde saklanır

static Anahtar Kelimesi - I



- Sınıf değişkenleri **static** anahtar kelimesiyle nitelendirilirler
- Nesne değişkenleri, her nesne için ayrı ayrı oluşturulduğundan dolayı *dinamik* ise, sınıf değişkenleri bu anlamda *statiktir*

```
class Citizen {  
    String tckn;  
    String firstName;  
    String lastName;  
    LocalDate dateOfBirth;  
    String placeOfBirth;  
    char sex;  
  
    String fatherName;  
    String motherName;  
  
    static String nationality;  
}
```

static Anahtar Kelimesi - II



- Tanıtım açısından sınıf değişkenlerinin nesne değişkenlerinden farkı, tipten önce **static** ile nitelendirilmeleridir
- Sınıf değişkenleri nesne değişkenleri gibi herhangi bir alt kapsamın dışında, genelde sınıfın başında tanıtırılır
- Kapsamları (scope), nesne değişkenleri gibi tüm sınıftır

static Anahtar Kelimesi - III



- **static** değişkenler nesnelerin ortak durumunu oluşturur
- **static** bir değişkenin tek bir kopyası dolayısıyla da tek bir değeri vardır, o da sınıfta saklanır, bu sebeple sınıf değişkeni de denir
- **static** değişkenlere ulaşmak için nesneye ihtiyaç yoktur, sınıf üzerinden, sınıfın ismiyle ulaşırlar
- Ulaşım için **.** notasyonu kullanılır

`ClassName.staticAttribute`

Sınıf Değişkeni - II



- Sınıf değişkenlerine, nesnelerinin daha doğrusu üzerinde bulunduğu sınıftan oluşturulan nesnelerin referansları üzerinden de ulaşmak mümkündür
- Ama aslolan sınıfın üzerinden ulaşmaktır

`reference.staticAttribute`

`ClassName.staticAttribute`

- Sınıf değişkenleri de, nesne değişkenleri gibi, varsayılan ilk duruma sahip olacak şekilde uygun değerlerle başlatılırlar

Sınıf Değişkeni - II



- Sınıf değişkenlerinin değeri tüm nesneler için aynıdır

```
static void main(String[] args){
    Citizen c1 = new Citizen();
    c1.tckn = "11111111111";
    c1.firstName = "Mehmet";
    c1.lastName = "Telli";

    Citizen c2 = new Citizen();
    c2.tckn = "22222222222";
    c2.firstName = "Selin";
    c2.lastName = "Merakli";

    Citizen.nationality = "Turkiye Cumhuriyeti";

    // Following line prints Turkiye Cumhuriyeti
    // Class access
    System.out.println("Nationality: " + Citizen.nationality);
    // Instance access
    System.out.println("Mehmet's nationality: " + c1.nationality);
    System.out.println("Selin's nationality: " + c2.nationality);
}
```



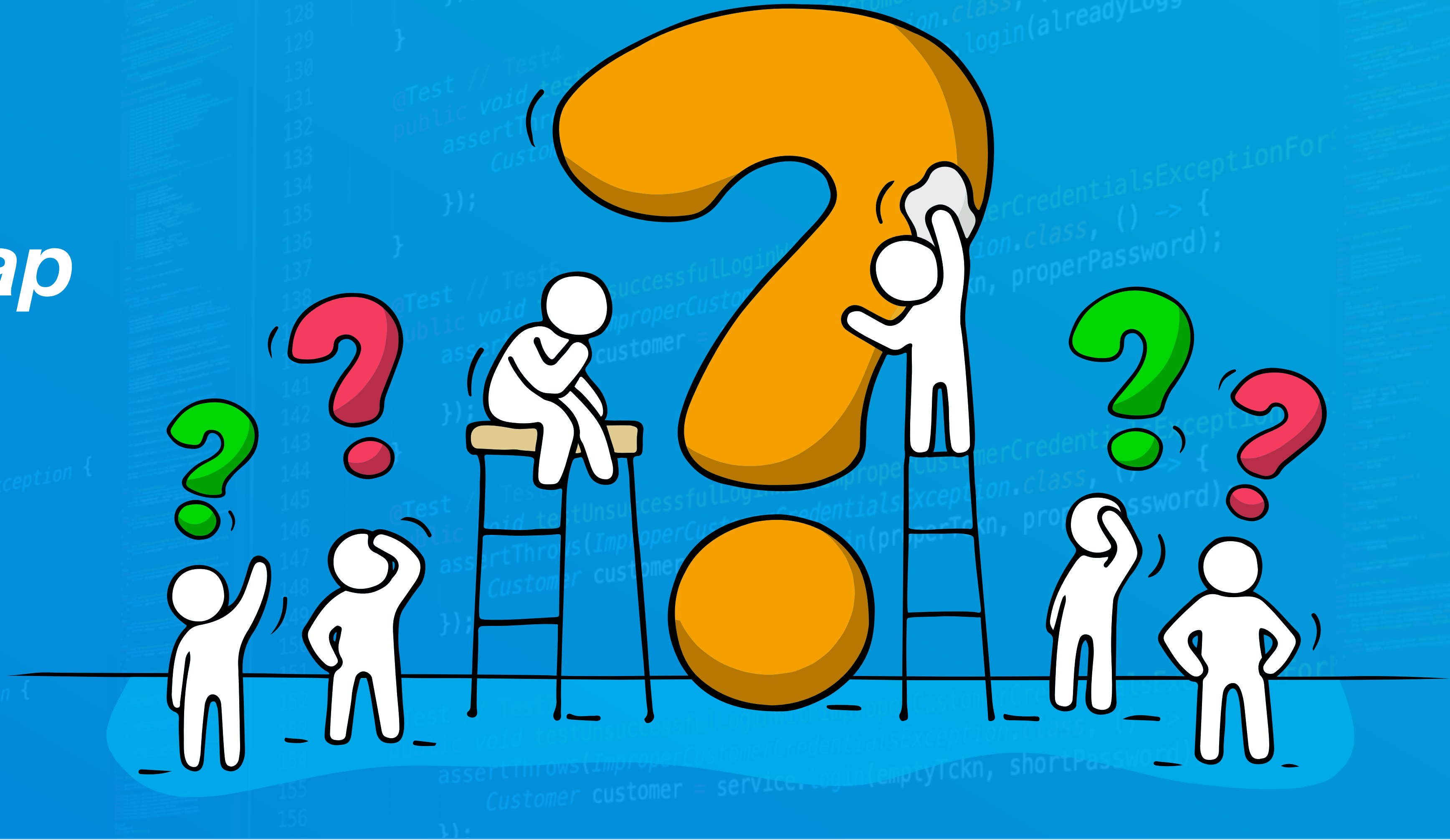
- Citizen

Üye Değişkenler



- Sınıf ve nesne değişkenlerine kısaca **üye değişkenler** (**member variables**) ya da **veri üyeleri** (**data members**) de denir

Soru ve Cevap Zamanı!





static Metotlar

static Metotlar - I



- Alanların yanında metotlar da **static** yapılabilir
- Metot tanımlanırken **static** niteleyicisi kullanılır
- **static** niteleyicisi dönüş tipinden önce olmalıdır
- **static** metotlar da **static** değişkenler gibi, sınıfın bir metodu yani sınıfın bir parçasıdır

```
class Citizen {  
    ...  
    static String getNationality(){  
        return nationality;  
    }  
}
```

static Metotlar - II



- **static** metotları çağırmak için nesneye ihtiyaç yoktur
- Ama **static** metotlar da alanlar gibi hem sınıfın hem de nesnelerinin üzerinden çağrılabilir
- Fakat aslolan sınıfın üzerinden ulaşmaktır

```
ClassName.staticMethod()
```

```
reference.staticMethod()
```

main as a static Metot



- En tipik **static** metot **main** metottur
- **main**, daima ilk çalışan metot olduğundan çağrılmak için bir nesneye ihtiyaç duymamalıdır

```
public static void main(String[] args)
```

```
public static void main(String ... args)
```

main as a Non-static Method - I



- Java 25 ile gelen özellik sayesinde `main` metod artık `static` olmak zorunda değildir

```
public void main(String[] args)
```

```
void main(String[] args)
```

```
void main()
```

- `static` olarak tanımlanmayan `main` metoda **nesne main metodu** (**instance main method**) denir
- Bu yenilikle `main` metodun artık `String` dizi argümanı alması ya da `public` olması da gerekmiyor

main as a Non-static Method - II



- Dolayısıyla `main` metot `static` olsun olmasın bir sınıf içinde ya da sınıf olmadan bir Java kaynak kod dosyasında tanımlanabilir

```
// InstanceMain.java
String s = "...";
Random r = new Random();

void main() {
    saySalam(...);
    sayHello(...);
}

void saySalam(String whom){
    ...
}

void sayHello(String whom){
    ...
}
```

```
// StaticMain.java
static String s = "...";
static Random r = new Random();

static void main() {
    saySalam(...);
    sayHello(...);
}

static void saySalam(String whom){
    ...
}

static void sayHello(String whom){
    ...
}
```

```
class GreetingWithInstanceMain{
    String s = "...";
    Random r = new Random();
    ...

    void main() {
        saySalam(...);
        sayHello(...);
    }

    void saySalam(String whom){
        ...
    }

    void sayHello(String whom){
        ...
    }
}
```

```
class GreetingWithStaticMain{
    static String s = "...";
    static Random r = new Random();
    ...

    static void main() {
        saySalam(...);
        sayHello(...);
    }

    static void saySalam(String whom){
        ...
    }

    static void sayHello(String whom){
        ...
    }
}
```

main Paketi



- `main` paketinde
- `InstanceMain` & `StaticMain`
- `GreetingWithInstanceMain` & `GreetingWithStaticMain`

static Alan ve Metotlara Erişim - I



- **static** alanlar ve metotlara hem sınıfın hem de nesnelerinin üzerinden ulaşılır
- Fakat **static** yapılara nesnelerin üzerinden ulaşmak, gerekmedikçe uygun değildir, çünkü böyle erişim **static** yapılara nesne değişkeni ve metodu izlenimi verir
- Bu yüzden IDE'ler genelde, **static** yapıları eğik olarak göstermekte ve nesne üzerinden erişimde uyarmaktadır

StaticDemo



- `StaticDemo`

static Alan ve Metotlara Erişim - II



- Nesne metotları, **static** olsun olmasın tüm değişken ve metotlara yani tüm üyelere (members) ulaşabilir
- Ama **static** metotlar sadece **static** değişkenlere doğrudan ulaşabilir, nesne değişkenlerine ulaşamaz
- Çünkü **static** metot zaten nesneyi varsaymadığından, ancak nesneyi varsaymayan yani **static** değişken ve metotlara ulaşması söz konusudur

static Alan ve Metotlara Erişim - II



- Hatırlayın, aynı kapsamda ve aynı isimde bir yerel bir de nesne değişkeni varsa, doğrudan erişimde yerel değişkene ulaşılıyordu
 - Nesne değişkenine ulaşmak için `this` gerekliydi
- Benzer şekilde aynı kapsamda ve aynı isimde bir yerel bir de sınıf değişkeni varsa, doğrudan erişimde yerel değişkene ulaşılır
- Sınıf değişkenine ulaşmak için sınıf ismi kullanılır

static Alan ve Metotlara Erişim - IV



- Bu durum, `this` kullanımından farklı değildir

```
class StaticDemo {  
    static int i = 10;  
    ...  
  
    static void setI(int i) {  
        StaticDemo.i = i;  
    }  
  
    static void aMethod() {  
        int i = 7;  
        System.out.println("Local i: " + i);  
        System.out.println("StaticDemo's i: " + StaticDemo.i);  
    }  
    ...  
}
```

static ve this



- **static** metotlar nesne olmadan çağrıldığından yani nesneyi var saymadığından **static** metotlarda **this** de yoktur!
- **static** metotlarda **this**'e ulaşmaya çalışmak derleme hatasıdır

StaticDemo & StaticAccess



- `StaticDemo`
- `StaticAccess`

main Paketi



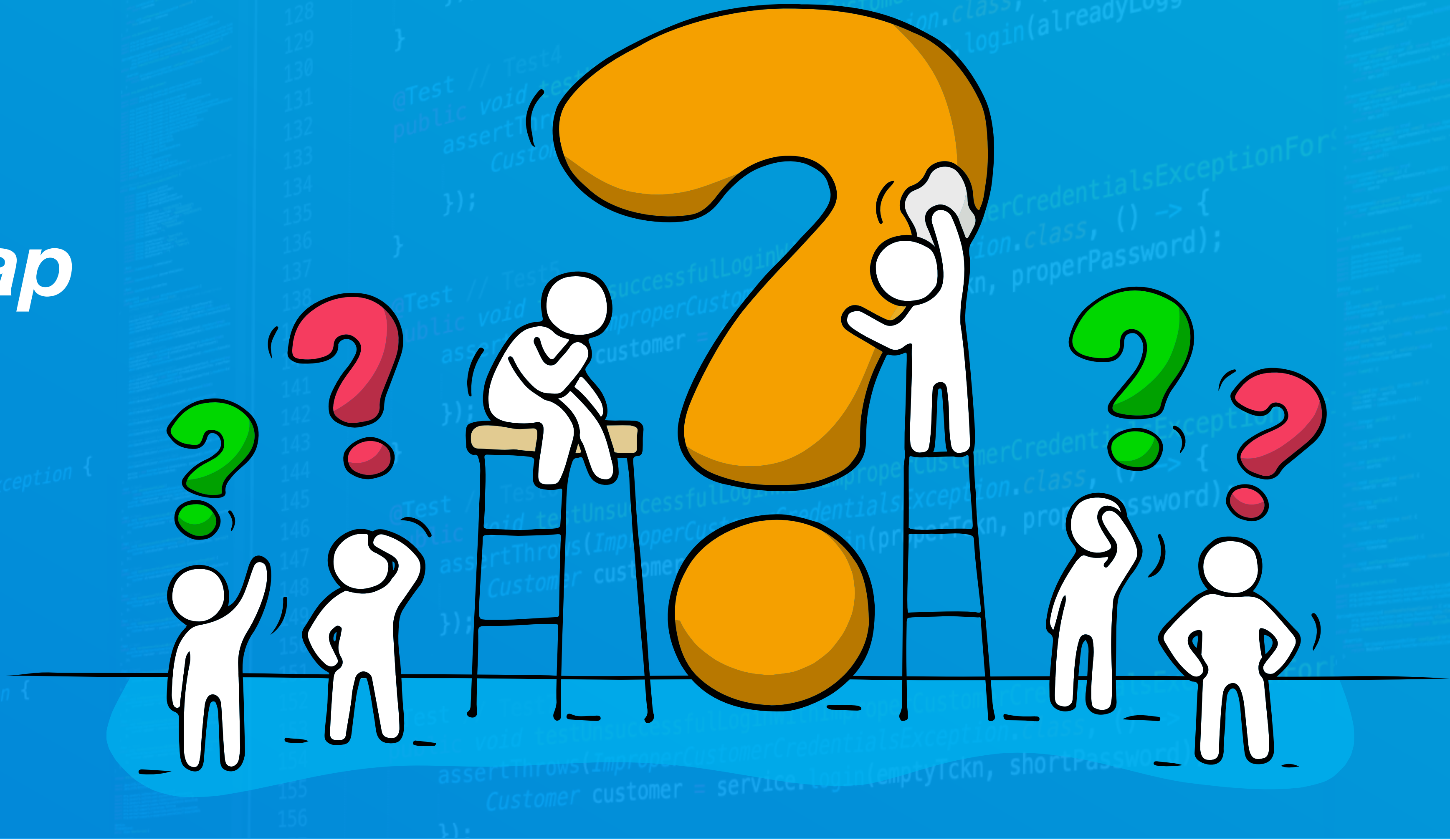
- `main` paketinde
- `InstanceMain` & `StaticMain`
- `GreetingWithInstanceMain` & `GreetingWithStaticMain`
- `main` metodun `static` olduğu durumda alanların ve metotlarının da `static` olmak zorunda kaldığını gözlemleyin

Üye Metotlar



- Sınıf ve nesne metotlarına kısaca **üye metotlar (member methods)** da dendiğini biliyoruz

Soru ve Cevap Zamanı!



Ne Zaman static?



Ne Zaman static? - I



- `static` ile ilgili en temel soru şudur: Ne zaman statik yapılar nesne yapılarına tercih edilmelidir?
- Nesne merkezli programlamada aslolan nesnedir, dolayısıyla her şeyin nesnenin parçası olarak düşünülmesi ve yapılmasıdır
- Ama `static` değişken ve metotların bazı hallerde kaçınılmaz olarak kullanılması gerekir

Ne Zaman static? - II



- `static` yapılar, nesnelere ihtiyaç duyulmadığı, daha doğru ifadeyle *nesnelerin anlam ifade etmediği veya gereksiz olduğu* durumları temsil ederler
- `main` metot nesne oluşturulmadan çağrılmalıdır
- `Math` sınıfındaki `PI` ve `E` değişkenleri ile matematiksel işlem yapan metotlarına nesne oluşturulmadan ulaşılabilmelidir
- Bu ve benzeri yapılarda nesne oluşturmak bazen anlamsız, `main` örneğindeki gibi bazen de imkansızdır



- MathDemo

Ne Zaman static? - II



- “Nesnelerin anlamsız ya da gereksiz” olması ne demektir?
- Nesnelerin anlamsız olması, *sınıfın farklı durumlara sahip nesnelerini oluşturmanın gerçeğe aykırı bir durum* olması demektir
 - **Math** sınıfının farklı duruma sahip nesnelerini oluşturmak anlamsızdır
- Dolayısıyla, nesneden nesneye değişen durumlar yerine tek bir durum söz konusudur, yani nesnesi olsa bile zaten sadece bir tane olabilir
- Böyle durumlarda sınıf, mümkün tek nesne gibi düşünülebilir

Ne Zaman static? - IV



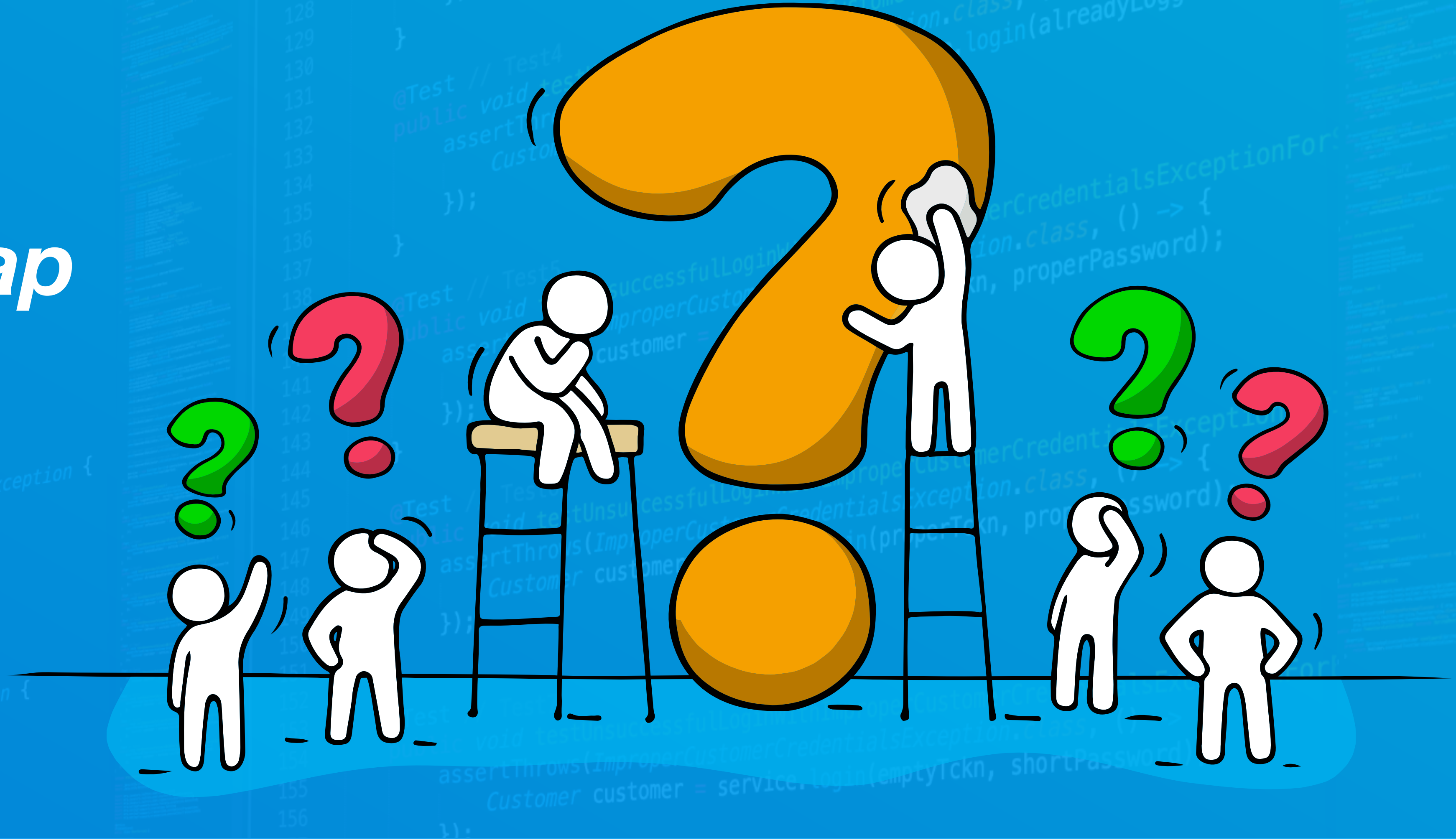
- Örneğin `Math` ya da `System` sınıfı bu türdendir, nesnelerinin olmasının anlamı yoktur
- `Customer`, `Account`, `Training`, `Student` vb. iş nesneleri (business objects) ise tam olarak bu durumun karşısında olan türden sınıflardır, daima birden fazla nesnesine ihtiyaç vardır
- Bu örneklerden görüldüğü gibi aslolan nesnedir yani nesne değişkenleri ve metotlarıdır; sınıf değişkenleri ve metotları özel durumlara özgüdür

Geometry



- `Geometry`
- Bu sınıftaki statik alan ve metotları inceleyin

Soru ve Cevap Zamanı!





Araçsal Sınıflar

Araçsal Sınıflar - I



- Bazen bazı sınıfların nesnelerini oluşturmak mümkün olsa bile ayırt edici durumları söz konusu olmadığından nesnelerini oluşturmak gereksizdir
- Tek olan sınıf, nesne gibi kullanılır
- Bu tür sınıflara **araçsal sınıf (utility class)** denir
- Araçsal sınıflardaki `static` metotlara **araçsal metot (utility method)** denir

Araçsal Sınıflar - II



- Araçsal sınıflar sıklıkla duruma sahip olmaz; yani sınıf değişkenleri bile olmaz
- Metotları da her seferinde bir işi yapmak için gerekli argümanları alıp sonucu geri döndürecek şekilde, durumsuz olarak (stateless) tasarlanır
- Metotlar, argümanlar dışında nesne değişkeni kullanmaz
- Eğer araçsal bir sınıf bir duruma sahipse, `static` olan metotların alanları kullanabilmesi için tüm alanları da `static` yapılır

Araçsal Sınıflar - III



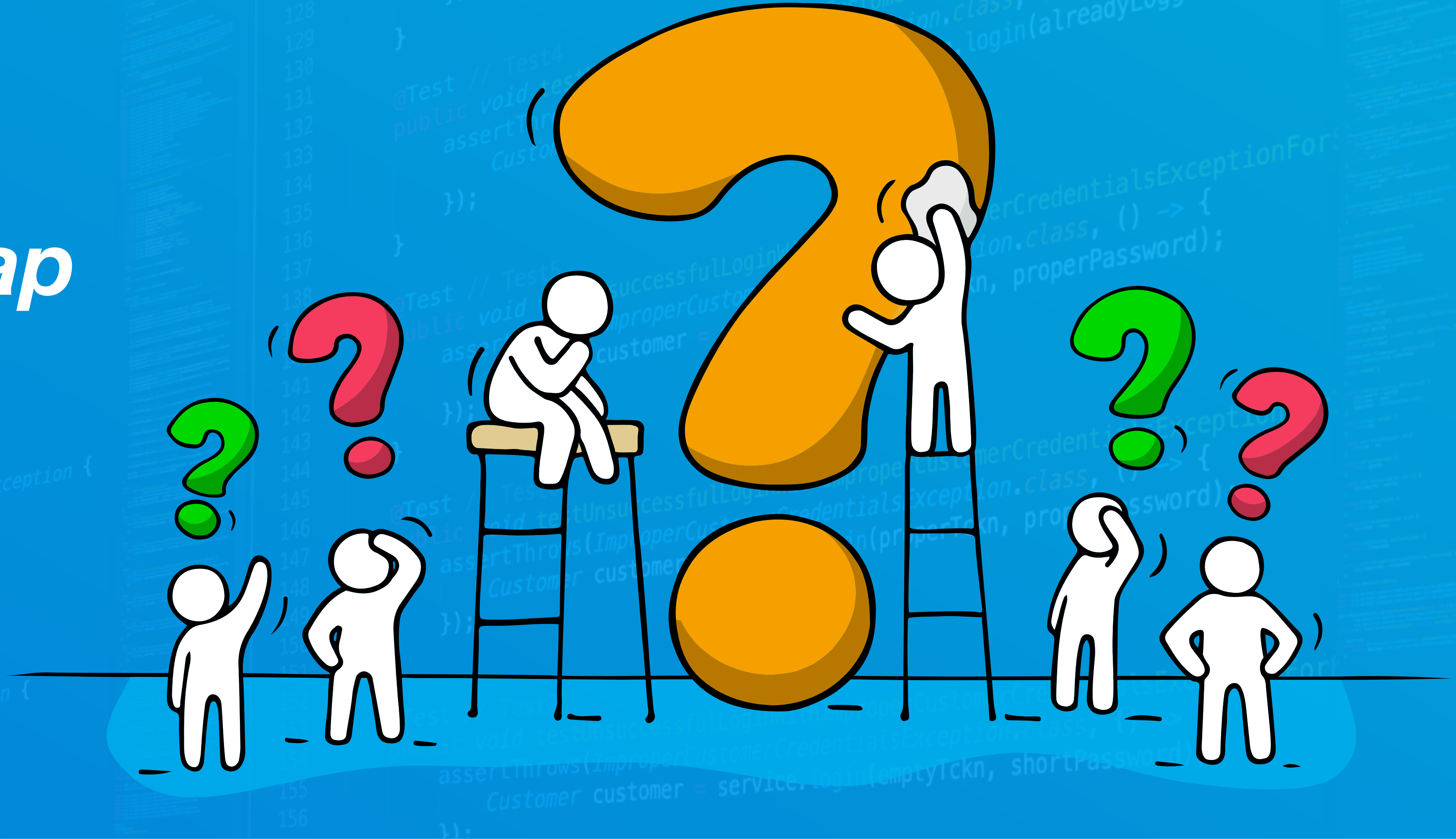
- Örneğin aşağıdaki sınıflar bu türdendir:
 - `java.lang` paketindeki `Math`
 - `java.lang` paketindeki `System`
 - `java.util` paketindeki `Arrays`
 - `java.util` paketindeki `Collections`

SystemDemo



- SystemDemo
- MathDemo

Soru ve Cevap Zamanı!



Ne Zaman static Değil?



Ne Zaman static Değil? - I



- Statik kullanımı, tamamen marjinal, özel durumlara has ve sıra dışı bir durumdur
- Aslolan daima nesnedir yani nesne değişkenleri ve metotlarıdır
- Çünkü nesneler problemimizi modellemeye yarar
- **static** değişken ve metotlar ise bu modelde özel durumlarda ortaya çıkar ve kullanımları ancak bu özel durumlara has olmalıdır

Ne Zaman `static` Değil? - II



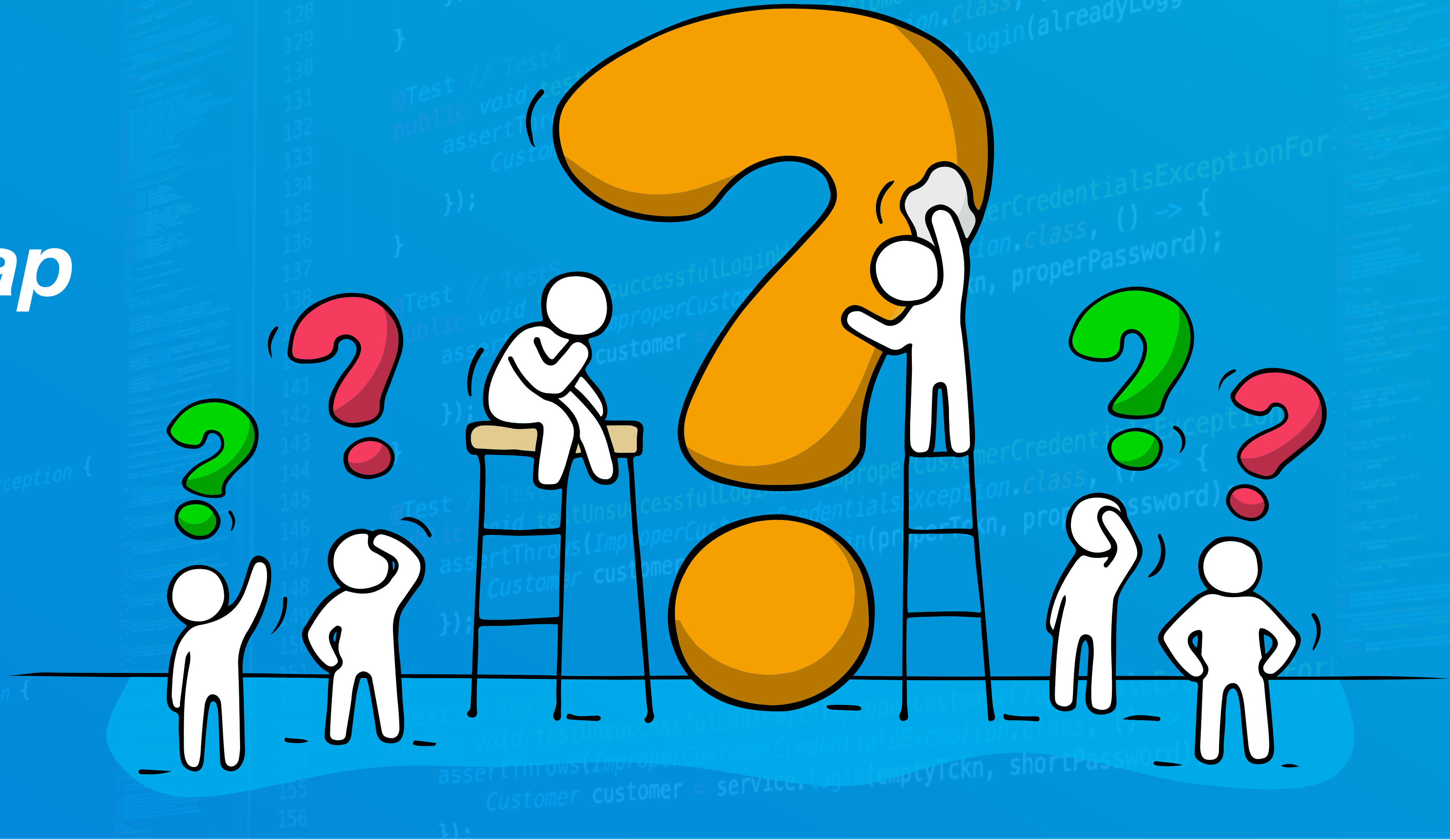
- Çok nesne oluşturma'nın gereksiz olduğu durumların çözümü genel olarak `static` değildir
- Bu durumun çözümü, gereksiz nesne oluşturmamaktır, çünkü gereksiz nesne oluşturmak zaten bellek problemlerine yol açar

Ne Zaman static Değil? - III



- Doğru `static` kullanımı için
 - Ya `Math` ve `System` örneğindeki gibi anlam olarak nesneyi düşünmek zor olmalıdır
 - Ya da araçsal sınıflardaki gibi birden fazla nesneye ihtiyaç olmamalıdır, sınıf tek nesne olarak kullanılmalıdır
 - Bu durumda değişken olarak da zaten ancak sınıf değişkenlerinden bahsedilir
 - Muhtemelen sabiteler de vardır

Soru ve Cevap Zamanı!





static
Bulaşıcıdır

static Bulaşıcıdır - I



- Aşırı miktarda `static` kullanımı kodunuzu nesne merkezli olmaktan çıkarır
- Çünkü nesnenin olmadığı varsayılarak kod yazıldığında, nesne oluşturmadan ilerlemek için daha çok `static` yapı kullanılır
- `static` metot zaten nesneyi varsaymadığından ancak nesne olmadan ulaşılacak yani `static` değişken ve metotlara ulaşabilir

static Bulaşıcıdır - II



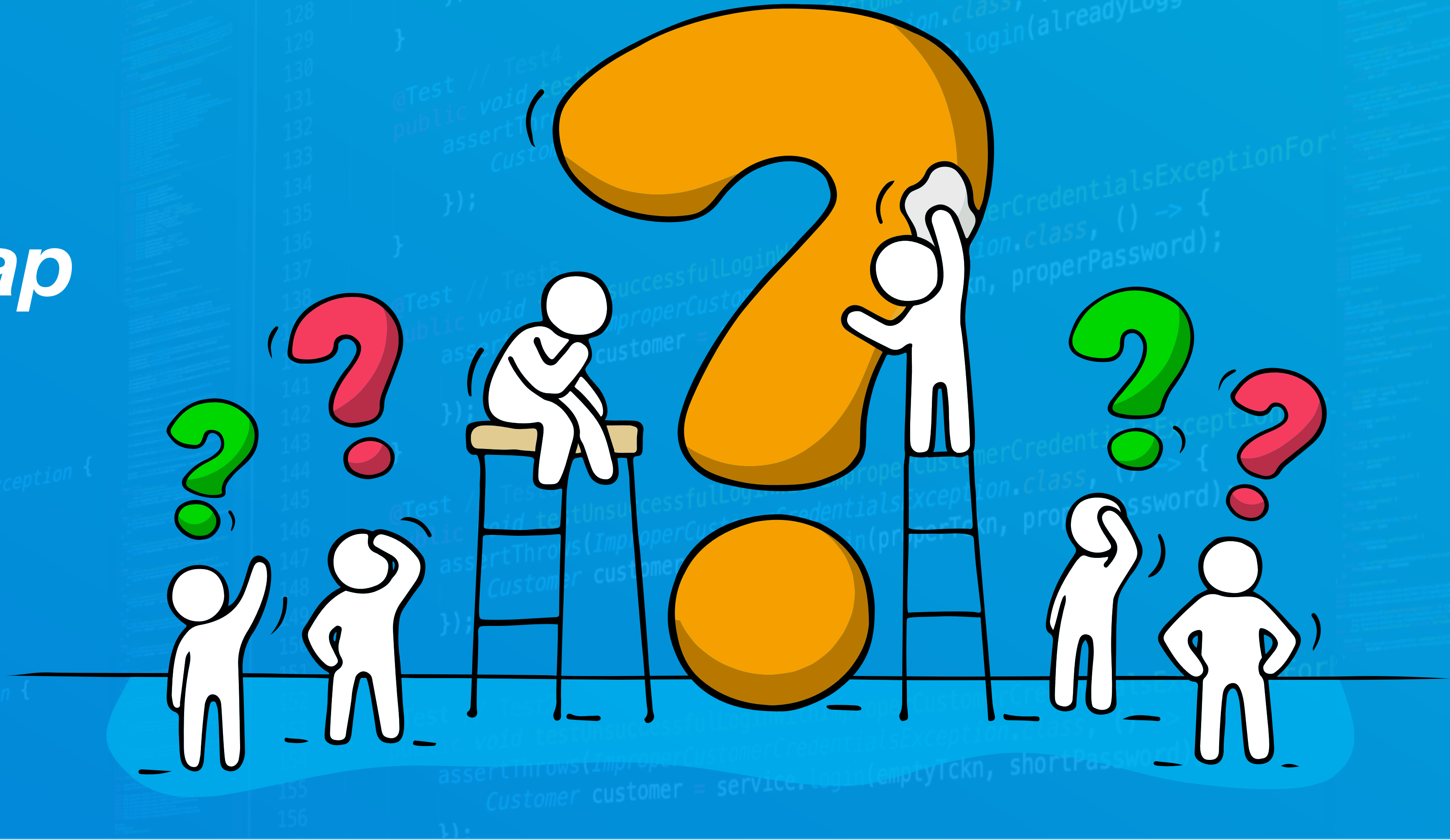
- Bu da **nesnesiz nesne-merkezli programlamaya** götürür:
- C görünümlü Java kodları!

PovertyOfNewbieJavaDeveloper



- `PovertyOfNewbieJavaDeveloper`
- Bu örnekte henüz nesne merkezli programlamayı anlamamış yeni bir Java programcısının nesne ile imtihanını ve bu imtihanda **static** ile nasıl başarısız bir çözüm geliştirdiğini gözlemleyeceksiniz
- **static** kullanımının bulaşıcılığını da gözlemleyin

Soru ve Cevap Zamanı!



Java'da Sabiteler



static ve final - Sabiteler - I



- `static` ve `final` kullanarak, sınıflar üzerinden erişilen ve değeri değişmeyen **sabite** (**constant**) oluşturulabilir
- Bu şekilde tanımlanan sabitelerin isminde *Camel Case* kullanılmaz, büyük harfler kullanılır ve kelimeler arasına `_` getirilir
- Bu şekilde sabiteler daha görünür kılınır

```
public static final double E = 2.7182818284590452354;  
public static final double PI = 3.14159265358979323846;
```

```
static final int MAX_UNIT_COUNT = 100;  
static final double STANDARD_LOAD_RATIO = 0.8;
```

static ve final - Sabiteler - II



- Bu sabiteler, yerel sabitelerden farklıdır çünkü bunlar `static`'tir ve bu yüzden de sınıfın üzerinden ulaşılır

```
double area = Math.PI * Math.pow(r, 2);
```

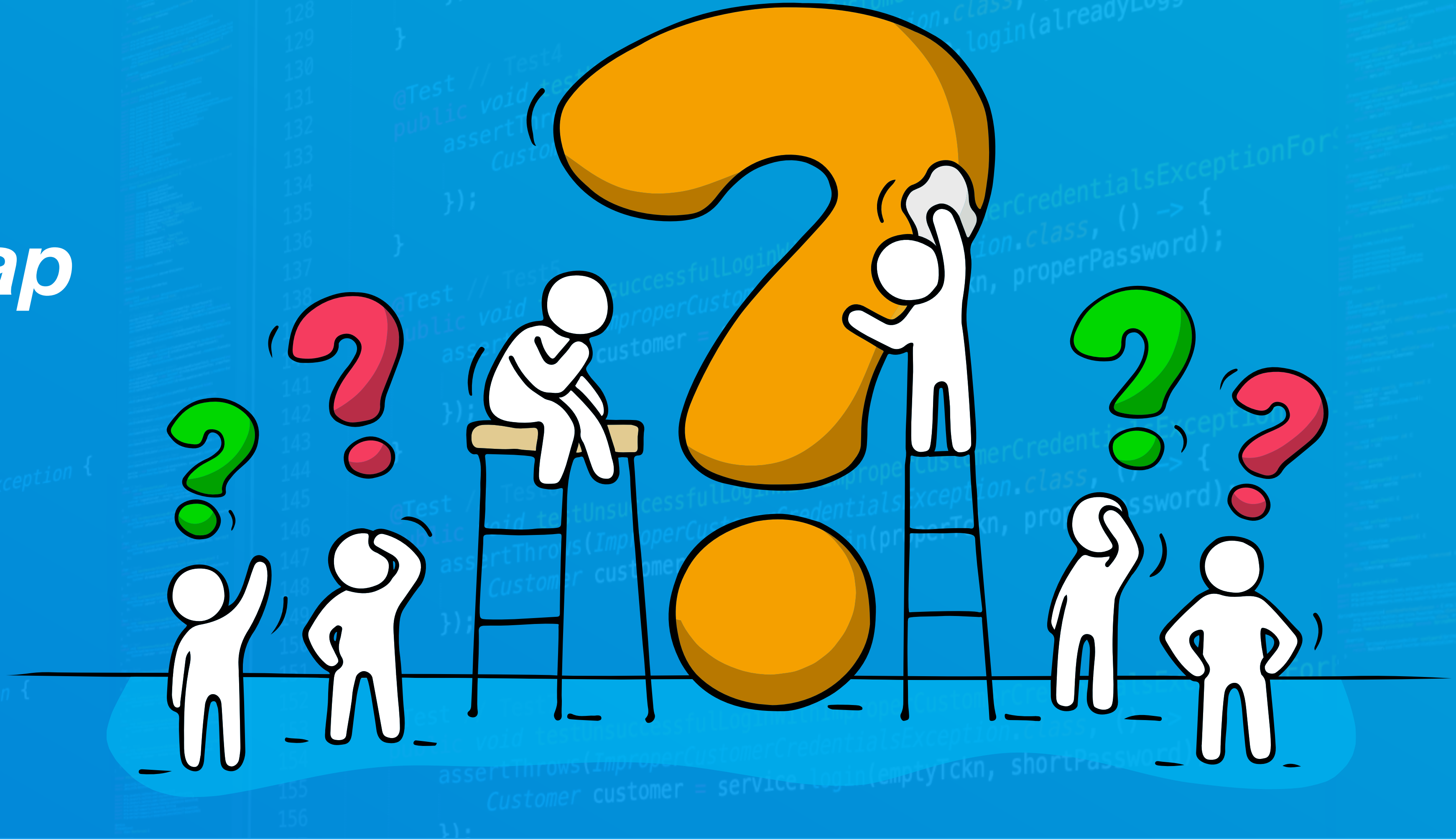
- `static` ve `final` olan sabitelerin bu şekilde yazılmaları için `public` olmalarına gerek yoktur
- Java'da sabiteler `static` ve `final` ile oluşturulduğundan `const` anahtar kelimesinin kullanım alanı yoktur

Tax & TaxCalculator



- `tax` paketinde `Tax & TaxCalculator`

Soru ve Cevap Zamanı!



Not



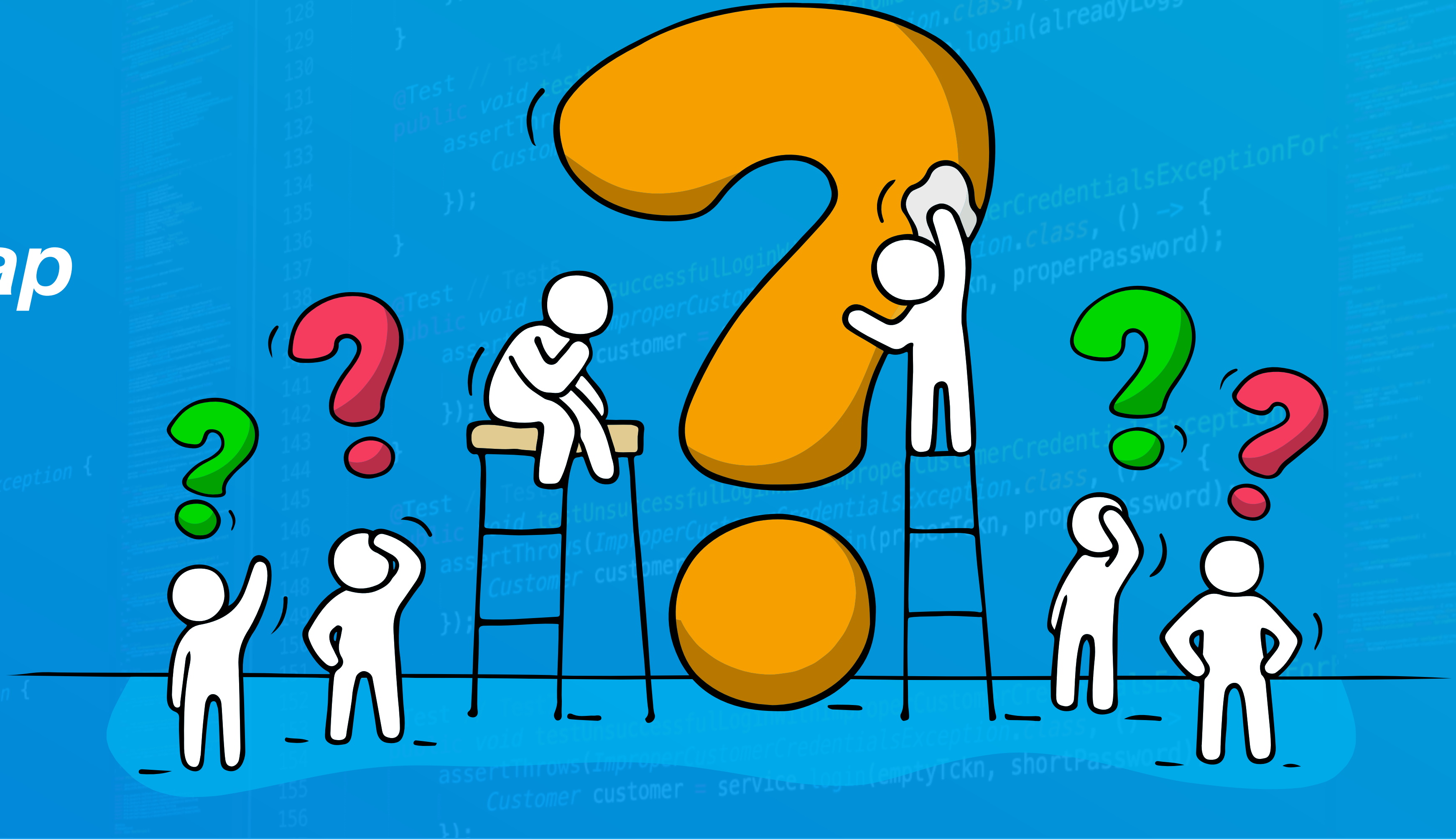
Kod Örneklerindeki Statik Metotlar



- Muhtemelen `static` ile ilgili tüm bu tartışmalardan sonra, şu ana kadar verilen kod örneklerinde neden hep `main` vb. `static` metotlar kullandığımız, anlaşılmıştır
- O andaki konuya odaklanıp, erkenden `static` tartışmasına girmemek için

```
class VarArgMethod {  
  
    public static void main(String... args) {  
  
        calculateAverage(4, 3);  
        calculateAverage(4, 3, 8);  
  
        int a[] = { 15, 25, 35, 45, 55, 65 };  
        calculateAverageWithArray(a);  
  
        int b[] = { 15, 25, 35, 45, 55, 65, 75, 86 };  
        calculateAverageWithArray(b);  
  
        average(); // Calls with no argument  
        average(1);  
        average(1, 2, 3, 4);  
        average(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);  
        average(0xAB);  
        average(b);  
  
        anotherAverage(2, true, 3, 4, 5);  
    }  
    ...  
}
```

Soru ve Cevap Zamanı!



Ödevler





1. Nesnelere ihtiyaç olmayan sınıflara örnekler verin
2. Aşağıdaki sınıfların nesnelerini oluşturmayı dolayısıyla varsa durum ve davranışlarının statik olup olmayacağını tartışın:
 - Aşk (Love)
 - Adalet (Justice)
 - Geometri (Geometry)
 - Tanrı (God)



3. Daha önce geçen `Point` sınıfını ile ilgili `Geometry` isimli sınıfı düşünün ve ona aşağıdaki metotları ekleyin:
- Geçilen iki `Point` nesnesi arasındaki mesafeyi hesaplayan metot
 - Geçilen bir `Point` nesnesinin (0, 0) noktasına olan uzaklığını hesaplayan metot
 - Bu metotların `static` olup olmayacaklarını tartışın
 - (0, 0) noktasını da bir `Point` olarak temsil etmek gerekirse onun da `static` olup olmayacağını tartışın



4. `static` kullanımına en güzel örneklerden birisi aşağıda verilmiştir:

- Bir sınıftan kaç tane nesne oluşturulduğunu nasıl bulursunuz?
- Bir sınıf geliştirin ve bu sınıfın herhangi bir kurucusunu çağırarak oluşturulan tüm nesneleri sayın
- Bu sayıyı tutacak değişkenin nesne için mi yoksa sınıf için mi olması gerekir?



5. `UnitConverter` isimli, farklı birimler arasında dönüşümler yapmak için kullanılacak bir sınıf düşünün. Bu sınıfın aşağıdaki gibi metotları olsun. Bu metotları `static` yapmayı tercih eder miydiniz? Bu sınıfın nesne değişkenleri olabilir mi? Neden?

a. `double celsiusToFahrenheit(double celsius)`

b. `double milesToKilometers(double miles)`

c. Test sınıfının `main` metodun bu metotları çağırın

Bundan Sonra?



Bundan Sonra?



- Bundan sonra başlatma bloklarını (initialization blocks) inceleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

